

Crypto & Stock Anomaly Detection System

Abdul Samad & Abdul Haseeb

DATABASE SYSTEMS LAB — PROJECT PROPOSAL

March 2025

1. OBJECTIVE OF THE PROPOSAL

The purpose of this proposal is to clearly explain the problem being addressed, the system to be developed, and the role of the database in solving it. The **Crypto & Stock Anomaly Detection System** aims to demonstrate how a well-designed relational database combined with a machine learning pipeline can detect unusual financial market behaviour in real time. The **primary focus is Database Design and Implementation** using MySQL — including normalized schema design, stored procedures, triggers, views, and complex SQL queries — all applied to a real-world, data-intensive financial domain. This project is submitted by **Abdul Samad and Abdul Haseeb**, BSSE 4th Semester, Group A.

2. PROJECT TEAM INFORMATION

Field	Details
Student 1	Abdul Samad
Student 2	Abdul Haseeb
Program	BSSE — Bachelor of Science in Software Engineering
Group	Group A — 4th Semester
Institution	Institute of Management Sciences (IMSciences)
Course	Database Systems Lab
Submission	March 2025

3. INTRODUCTION & BACKGROUND

Financial markets — particularly cryptocurrency markets — generate millions of price records every day, creating massive structured datasets well-suited for relational database management. However, detecting abnormal behaviour within this data — such as sudden price crashes, manipulation spikes, or unusual trading volumes — remains a significant challenge. Traditional rule-based alert systems are too rigid to capture complex multi-dimensional anomalies. By storing market data in a normalized **MySQL** database and applying machine learning on top of it, it becomes possible to automatically flag suspicious patterns and give analysts an intelligent, data-driven decision support tool. This project sits at the intersection of database engineering, financial data analysis, and applied AI.

4. PROBLEM STATEMENT

What problem exists?

Crypto and stock markets experience sudden abnormal price movements — spikes, crashes, and volume surges — caused by manipulation, flash crashes, and algorithmic errors that go undetected by traditional tools.

Who is affected?

Retail investors, financial analysts, and trading platforms that rely on accurate market data to make informed and timely decisions.

Current inefficiencies:

Existing tools rely on manual monitoring or simple price-threshold alerts, which are slow, generate false positives, and cannot detect multi-dimensional patterns across price, volume, and volatility simultaneously.

Why a database solution?

A relational database provides the structured storage, fast querying, and data integrity needed to log, retrieve, and analyse large volumes of OHLCV records — making it the ideal backbone for an automated anomaly detection system.

5. PROPOSED SOLUTION

The proposed system is the **Crypto & Stock Anomaly Detection System** — a full-stack web application that: (1) ingests historical OHLCV data from CSV datasets and live market APIs; (2) stores all data in a fully normalized MySQL database with 6 relational tables; (3) runs an **Isolation Forest** ML model to detect anomalies; (4) persists detected anomalies back into the database via stored procedures; and (5) displays everything on an interactive **Flask** dashboard with real-time charts, severity-classified alerts, and user authentication. This approach is effective because it combines robust RDBMS design with modern unsupervised ML — all data stays structured and queryable within MySQL.

6. OBJECTIVES OF THE SYSTEM

- Design a fully normalized **(3NF) MySQL schema** with 6 relational tables: assets, price_history, anomalies, alerts, users, and model_logs.
- Build a Python data pipeline to bulk-load historical CSV data and fetch live OHLCV data via **CoinGecko / Yahoo Finance APIs** into MySQL.
- Train and deploy an **Isolation Forest** ML model to detect price spikes, crashes, volume surges, and volatility anomalies in the stored data.
- Persist every detected anomaly — with severity score, type, and timestamp — back into MySQL using a **stored procedure**.
- Develop a **Flask web dashboard** with Chart.js visualizations, anomaly alert table, and real-time notification system.
- Implement **user authentication** with role-based access control (Admin, Analyst, Viewer) for secure dashboard access.

7. SCOPE OF THE PROJECT

✓ IN SCOPE

- 10 crypto + 5 stock assets monitored
- Historical data (2020–2025) + live API feed
- Isolation Forest ML anomaly detection
- MySQL — full CRUD, procedures & triggers
- Flask web dashboard with user login
- Severity-classified anomaly alert system
- Stored procedures, triggers and SQL views

✗ OUT OF SCOPE

- Automated live trading / order execution
- Mobile application development
- Deep learning or neural network models
- Payment gateway or wallet integrations
- Derivatives, options or futures markets
- Multi-language (non-English) UI support

8. ROLE OF DATABASE SYSTEM

What data will be stored:

Asset master records, full OHLCV price time-series (millions of rows), ML anomaly detection results with severity scores, system alert logs, user accounts, and ML model audit trails.

Why a database is necessary:

The system deals with large volumes of structured numerical data that must be stored reliably, queried efficiently, and kept consistent across multiple application components. MySQL provides ACID compliance, foreign key constraints, and indexing that make this feasible.

Operations performed:

Full CRUD on all 6 tables; bulk INSERT of historical CSV data; stored procedures for saving anomalies and logging ML runs; triggers for auto-generating alerts; views for dashboard aggregations; complex multi-table JOIN queries for reporting.

9. SYSTEM FEATURES / FUNCTIONAL REQUIREMENTS

- **User Authentication:** Secure login with bcrypt password hashing and session management.
- **Role-Based Access Control:** Admin, Analyst, and Viewer roles with different permissions.
- **Data Ingestion:** Bulk-load historical CSV data and fetch live prices via CoinGecko API.
- **Anomaly Detection Engine:** Isolation Forest ML model runs on stored price data and flags anomalies.
- **Anomaly Alert System:** Auto-generated alerts via MySQL trigger when new anomaly is inserted.
- **Interactive Price Charts:** Chart.js line charts with anomaly markers overlaid on price history.

- **Anomaly Management Table:** Filter, sort, and confirm anomalies by asset, type, and severity.
- **Asset Summary Dashboard:** Summary cards showing total anomalies, critical count, last detection.
- **ML Model Audit Log:** Every model run logged with records scanned, anomalies found, and duration.
- **SQL Reporting Queries:** Pre-built aggregation queries for lab report — JOINS, GROUP BY, views.

10. PRELIMINARY DATA DESIGN

The system uses **6 relational tables** in 3rd Normal Form (3NF). Key relationships: *assets* → *price_history* (one asset has many price records); *price_history* → *anomalies* (one price record can trigger one anomaly); *anomalies* → *alerts* (one anomaly generates one alert); *users* → *model_logs* (a user can trigger many model runs).

Table	Primary / Foreign Keys	Purpose
assets	asset_id (PK), symbol, name, asset_type	Crypto / stock master list
price_history	price_id (PK), asset_id (FK), timestamp, open, high, low, close, volume	OHLCV time-series data
anomalies	anomaly_id (PK), asset_id (FK), price_id (FK), score, anomaly_type, severity	ML detection results
alerts	alert_id (PK), anomaly_id (FK), message, sent_at	Auto-generated notifications
users	user_id (PK), username, password_hash, role	Dashboard authentication
model_logs	log_id (PK), run_at, model_version, records_scanned	ML audit trail

11. EXISTING SYSTEMS / COMPARATIVE ANALYSIS

Existing financial anomaly tools — such as Bloomberg Terminal alerts, TradingView indicators, and CoinMarketCap trackers — rely on **simple threshold rules** (e.g. price changed by X%). They cannot detect complex multi-dimensional patterns and require manual configuration per asset.

Feature	Existing Tools	Our System
Detection Method	Rule-based thresholds	Isolation Forest ML
Data Storage	Proprietary / cloud	MySQL — fully local
Multi-dimensional	No	Yes (price + volume + volatility)
Custom Queries	Limited	Full SQL — JOINS, views, procedures
Cost	Expensive subscriptions	Free / open-source stack
ER Diagram / DB	No access to schema	Full normalized RDBMS design

12. PROPOSED TECHNOLOGY STACK

Database :	MySQL 8.0 — normalized schema, stored procedures, triggers, views, indexes
Backend :	Python 3.11 · Flask — REST routes, ML inference, DB connection layer
ML :	scikit-learn (Isolation Forest) · pandas · numpy · matplotlib
Frontend :	HTML5 · Bootstrap 5 · Chart.js — real-time dashboard & anomaly alerts
Data :	CoinGecko API · Yahoo Finance API · Kaggle CSV historical datasets
Dev Tools:	VS Code · MySQL Workbench · Postman · Git & GitHub

13. EXPECTED OUTCOMES

- A fully functional **MySQL database** with 6 normalized tables, stored procedures, triggers, and views — demonstrating mastery of relational database design principles.
- An **Isolation Forest ML model** achieving 85%+ precision in detecting genuine price anomalies across 15 monitored crypto and stock assets.
- A complete **ER diagram**, data dictionary, and schema documentation showing all table relationships and normalization.
- A live **Flask web dashboard** displaying real-time price charts, anomaly severity alerts, and a user-managed notification log.
- A **SQL query report** showcasing advanced queries — multi-table JOINS, GROUP BY aggregations, stored procedure calls, and view usage.
- Full project documentation and system architecture diagram ready for final submission and viva.